Nombre: ..... Curso: ..... Fecha: .....

INTERPRETE PYTHON

Objetivos	Utiliza sintaxis del interprete Python desde el pseudocodigo.
Contenidos	Algoritmos : Del Pseudocódigo a Script Python
Habilidades	Aplicación – Análisis
Instrucciones	• Para cada ejercicio, colocar el desarrollo en la misma hoja. • Se reciben trabajo con esta hoja, y las respuestas debe ir incluido el enunciado. .

Interprete Python

Python es un lenguaje de programación poderoso y fácil de aprender. Cuenta con estructuras de datos eficientes y de alto nivel y un enfoque simple pero efectivo a la programación orientada a objetos. La elegante sintaxis de Python y su tipiado dinámico, junto con su naturaleza interpretada, hacen de éste un lenguaje ideal para scripting y desarrollo rápido de aplicaciones en diversas áreas y sobre la mayoría de las plataformas.

Salida por pantalla: la función print()[1]

En Informática, la "salida" de un programa son los datos que el programa proporciona al exterior. Aunque en los incios de la informática la salida más habitual era una impresora, hace muchos años que el dispositivo de salida más habitual es la pantalla del ordenador.

En los programa, para mostrar texto o variables hay que utilizar la función print().
La función print() permite mostrar texto en pantalla. El texto a mostrar se escribe como argumento de la función:

```
print("Hola")

>> Hola
```

Las cadenas se pueden delimitar tanto por comillas dobles (") como por comillas simples (').

```
print('Hola')
>> Hola
```

Nota: En estos apuntes se utilizan normalmente comillas dobles.

Entrada por teclado: la función input()

La función input() permite obtener texto escrito por teclado. Al llegar a la función, el programa se detiene esperando que se escriba algo y se pulse la tecla Intro, como muestra el siguiente ejemplo:

```
print("¿Cómo se llama?")
nombre = input()
print("Me alegro de conocerle,", nombre)

 ¿Cómo se llama?
Pepe
Me alegro de conocerle, Pepe
```

En el ejemplo anterior, el usuario escribe su respuesta en una línea distinta a la pregunta porque Python añade un salto de línea al final de cada print().

Si se prefiere que el usuario escriba su respuesta a continuación de la pregunta, se podría utilizar el argumento opcional end en la función print(), que indica el carácter o caracteres a utilizar en vez del salto de línea. Para separar la respuesta de la pregunta se ha añadido un espacio al final de la pregunta.

Una solución, más compacta, es aprovechar que a la función input() se le puede enviar un argumento que se escribe en la pantalla (sin añadir un salto de línea):

```
nombre = input("¿Cómo se llama? ")
print("Me alegro de conocerle,", nombre)

 ¿Cómo se llama? Pepe
Me alegro de conocerle, Pepe
```

Estructuras de control: condicionales y bucles

Son parte fundamental de cualquier lenguaje. Sin ellas, las instrucciones de un programa sólo podrán ejecutarse en el orden en que están escritas (orden secuencial). Las estructuras de control permiten modificar este orden. Hay dos categorías de estructuras de control:

---

1 http://www.mclibre.org/consultar/python/lecciones/python_salida_pantalla.html

Condicionales o bifurcaciones: permiten que se ejecuten conjuntos distintos de instrucciones, en función de que se verifique o no determinada condición.

Bucles o repeticiones: permiten que se ejecute repetidamente un conjunto de instrucciones, bien un número pre-determinado de veces, o bien hasta que se verifique una determinada condición.

La sentencia if[2]

Tal vez el tipo más conocido de sentencia sea el **`if`**. Por ejemplo:

```
>>> x = int(input("Ingresa un entero, por favor: ")
)
Ingresa un entero, por favor: 42
>>> if x < 0:
...      x = 0
...      print('Negativo cambiado a cero')
... elif x == 0:
...      print('Cero')
... elif x == 1:
...      print('Simple')
... else:
...      print('Más')
...
'Mas'
```

Puede haber cero o más bloques **`elif`**, y el bloque **`else`** es opcional. La palabra reservada '**`elif`**' es una abreviación de 'else if', y es útil para evitar un sangrado excesivo. Una secuencia **`if`** ... **`elif`** ... **`elif`** ... sustituye las sentencias `switch` o `case` encontradas en otros lenguajes.

La sentencia for

La sentencia **`for`** en Python itera sobre los ítems de cualquier secuencia (una lista o una cadena de texto), en el orden que aparecen en la secuencia.

Por ejemplo:

```
>>> # Midiendo cadenas de texto
... palabras = ['gato', 'ventana', 'defenestrado']
>>> for p in palabras:
...     print(p, len(p))
...
gato 4
ventana 7
defenestrado 12
```

La función range()

Si se necesita iterar sobre una secuencia de números, es apropiado utilizar la función integrada **`range()`**, la cual genera progresiones aritméticas:

```
>>> for i in range(5):
...     print(i)
...

0
1
2
3
4
```

El valor final dado nunca es parte de la secuencia; `range(10)` genera 10 valores, los índices correspondientes para los ítems de una secuencia de longitud 10.
Es posible hacer que el rango empiece con otro número, o especificar un incremento diferente (incluso negativo; algunas veces se lo llama 'paso'):

```
range(5, 10)
   5 through 9

range(0, 10, 3)
   0, 3, 6, 9
```

Bucles WHILE[3]

En Python tenemos una palabra reservada llamada "while" que nos permite ejecutar ciclos, o bien secuencias periódicas que nos permiten ejecutar código múltiples veces.

El ciclo while nos permite realizar múltiples iteraciones basándonos en el resultado de una expresión lógica que puede tener como resultado un valor verdadero o falso (true o false).

```
print "\nWhile controlado con Conteo"
print "===========================\n"

print "Un ejemplo es un sumador numérico hasta 10,
\ncomo se muestra a continuación:\n"

suma = 0
numero = 1
while numero <= 10:
    suma = numero + suma
    numero = numero + 1
print "La suma es " + str(suma)
```

[2] http://docs.python.org.ar/tutorial/3/controlflow.html

[3] http://entrenamiento-python-basico.readthedocs.io/es/latest/leccion4/bucle_while.html

Operadores relacionales

Los valores booleanos son además el resultado de expresiones que utilizan operadores relacionales (comparaciones entre valores)[4]:

Operador	Descripción	Ejemplo
==	¿son iguales a y b?	r = 5 == 3 # r es False
!=	¿son distintos a y b?	r = 5 != 3 # r es True
<	¿es a menor que b?	r = 5 < 3 # r es False
>	¿es a mayor que b?	r = 5 > 3 # r es True
<=	¿es a menor o igual que b?	r = 5 <= 5 # r es True
>=	¿es a mayor o igual que b?	r = 5 >= 3 # r es True

Operadores aritméticos

Los valores numéricos son además el resultado de una serie de operadores aritméticos y matemáticos:

Operador	Descripción	Ejemplo
+	Suma	r = 3 + 2 # r es 5
–	Resta	r = 4 – 7 # r es -3
-	Negación	r = -7 # r es -7
*	Multiplicación	r = 2 * 6 # r es 12
**	Exponente	r = 2 ** 6 # r es 64
/	División	r = 3.5 / 2 # r es 1.75
//	División entera	r = 3.5 // 2 # r es 1.0
%	Módulo	r = 7 % 2 # r es 1

Puede que tengáis dudas sobre cómo funciona el operador de módulo, y cuál es la diferencia entre división y división entera.
El operador de módulo no hace otra cosa que devolvernos el resto de la división entre los dos operandos.
En el ejemplo, 7 / 2 sería 3, con 1 de resto, luego el módulo es 1.
La diferencia entre división y división entera no es otra que la que indica su nombre. En la división el resultado que se devuelve es un número real, mientras que en la división entera el resultado que se devuelve es solo la parte entera.

Bibliografía de referencia

- http://www.mclibre.org/consultar/python
- http://entrenamiento-python-basico.readthedocs.io/es/latest/
- http://docs.python.org.ar/tutorial/3/

Ejercicios

Interpretar los siguientes pseudocódigos, utilizando los contenidos descritos anteriormente escribir el pseudocódigo en script Python y el diagrama de flujo estructurado (DFE) en la hoja de desarrollo.

```
Algoritmo Triangulo
        Real<-num1
        Real=num2
        Escribir "escriba la medida de los catetos"
        Leer num1
        Escribir "escriba la medida del otro cateto"
        Leer num2
        resul=num1*num1
        resul2=num2*num2
        resul3=resul+resul2
        escribir "la hipotenusa de este triangulo es ", rc(resul3)
FinAlgoritmo
```

```
Algoritmo DeKiloaByte
        Real=num1
        Real=num2
        Escribir "Hola estimado, escriba los
kilobytes que quiera transformar"
        Leer num1
        resul=num1/1000
        Escribir "Hola caballero, son ", resul,"
megabytes"

FinAlgoritmo
```

1. Escriba un algoritmo que lea dos números, calculando y escribiendo el valor de su suma, resta, producto y división.
2. Escribir un algoritmo que calcule el cociente y el residuo dados dos números enteros.
3. Escribir un algoritmo que calcule la hipotenusa de un triángulo rectángulo.
4. Escribir un algoritmo que permita convertir un número dado en kilobytes a megabytes.
5. Escriba un algoritmo que permita calcular el área y perímetro de un círculo solicitando el radio (Defina una variable pi = 3,141592).

[4] http://entrenamiento-python-basico.readthedocs.io/es/latest/leccion3/index.html#operadores-relacionales